目标

1）ThreadLocal 它是什么

2）ThreadLocal 是干什么的

3）ThreadLocal 底层数据结构

4）ThreadLocal 如何解决 hash 冲突

5）ThreadLocal 扩容因子是 2/3，hashmap 的扩容因子是 0.75

6）java 中有哪几种引用

7）ThreadLocalMap 中的 key 是什么引用，为什么要用这种引用类型

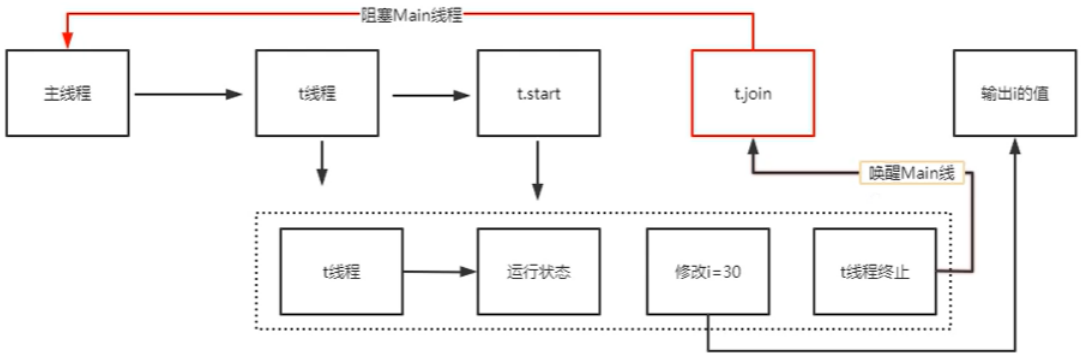
8）怎么解决内存泄漏呢？

1 死锁的基本条件

- 互斥：共享资源 X 和 Y 只能被一个线程占用；
- 占有且等待：线程 T1 已经取得共享资源 X，在等待共享资源 Y 的时候，不释放共享资源 X；
- 不可抢占：其他线程不能强行抢占线程 T1 占有的资源；
- 循环等待：线程 T1 等待线程 T2 占有的资源，线程 T2 等待线程 T1 占有的资源，就是循环等待。
- 这四个条件必须同时满足才会出现死锁

2 Tread.join

Thread.join，它的作用其实就是让线程的执行结果对后续线程的访问可见。实现原理是进行 wait 阻塞等待，等待 thead 执行完成，来保证在 thread 里面的更改可见



thread.join 底层调用的是当前线程实例的 wait 方法，然后当我们的当前线程执行完之后，在 JVM 底层会调用 thread.notifyAll，就是唤醒所有阻塞在当前线程实例的线程。

3 ThreadLocal

作用：ThreadLocal 用作保存每个线程独享的对象，为每个线程都创建一个副本，这样每个线程都可以修改自己所拥有的副本,而不会影响其他线程的副本，确保了线程安全（线程隔离机制）

3.1 使用场景

3.1.1 场景 1

这种场景通常用于保存线程不安全的工具类，典型的需要使用的类就是 **SimpleDateFormat**。

在这种情况下，每个 Thread 内都有自己的实例副本，且该副本只能由当前 Thread 访问到并使用，相当于每个线程内部的本地变量，这也是 ThreadLocal 命名的含义。因为每个线程独享副本，而不是公用的，所以不存在多线程间共享的问题。

代码见 scene1

在 demo01 中们给每个线程都创建了 SimpleDateFormat 对象，他们之间互不影响，代码是可以正常执行的。

在 demo02 中可以看出，我们用了一个 16 线程的线程池，并且给这个线程池提交了 1000 次任务。每个任务中它做的事情和之前是一样的，还是去执行 date 方法，

帅气的景天

并且在这个方法中创建一个 simpleDateFormat 对象。刚才所做的就是每个任务都创建了一个 simpleDateFormat 对象，也就是说，1000 个任务对应 1000 个 simpleDateFormat 对象，但是如果任务数巨多怎么办？这么多对象的创建是有开销的，并且在使用完之后的销毁同样是有开销的，同时存在在内存中也是一种内存的浪费。我们可能会想到，要不所有的线程共用一个 simpleDateFormat 对象？但是 simpleDateFormat 又不是线程安全的，我们必须做同步，比如使用 synchronized 加锁。到这里也许就是我们最终的一个解决方法。但是使用 synchronized 加锁会陷入一种排队的状态，多个线程不能同时工作，这样一来，整体的效率就被大大降低了。

在 demo04 中引入 ThreadLocal，对这种场景，ThreadLocal 再合适不过了，ThreadLocal 给每个线程维护一个自己的 simpleDateFormat 对象，这个对象在线程之间是独立的，互相没有关系的。这也就避免了线程安全问题。与此同时，simpleDateFormat 对象还不会创造过多，线程池一共只有 16 个线程，所以需要 16 个对象即可。

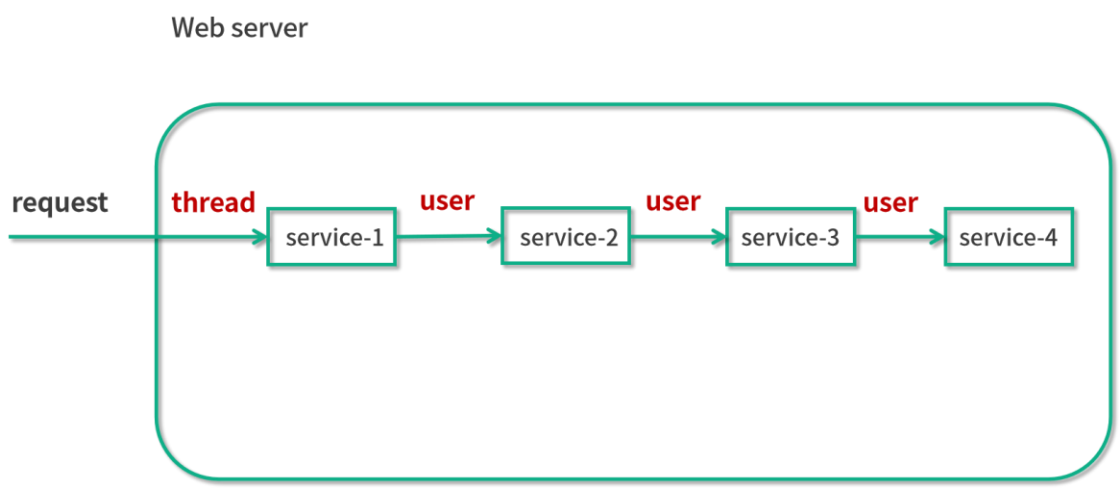
3.1.2 场景 2

每个线程内需要保存类似于全局变量的信息（例如在拦截器中获取的用户信息），可以让不同方法直接使用，避免参数传递的麻烦却不想被多线程共享（因为不同线程获取到的用户信息不一样）。

例如，用 ThreadLocal 保存一些业务内容（用户权限信息、从用户系统获取到的用户名、用户 ID 等），这些信息在同一个线程内相同，但是不同的线程使用的业务内容是不相同的。

在线程生命周期内，都通过这个静态 ThreadLocal 实例的 get() 方法取得自己 set 过的那个对象，避免了将这个对象（如 user 对象）作为参数传递的麻烦。

比如说我们是一个用户系统，那么当一个请求进来的时候，一个线程会负责执行这个请求，然后这个请求就会依次调用 service-1()、service-2()、service-3()、service-4()，这 4 个方法可能是分布在不同的类中的。



代码见 ThreadLocalDemo05

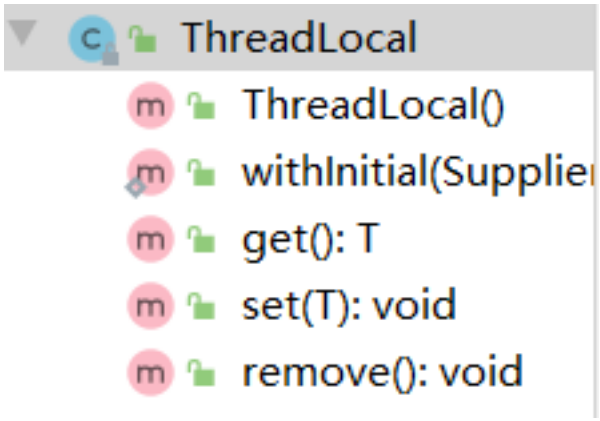
3.1.3 局限性

ThreadLocal 为解决多线程程序的并发问题提供了一种新的思路。但是 ThreadLocal 也有局限性，我们来看看阿里规范：

15. 【参考】ThreadLocal 无法解决共享对象的更新问题，ThreadLocal 对象建议使用 static 修饰。这个变量是针对一个线程内所有操作共享的，所以设置为静态变量，所有此类实例共享此静态变量 ，也就是说在类第一次被使用时装载，只分配一块存储空间，所有此类的对象(只要是这个线程内定义的)都可以操控这个变量。

3.2 实现原理

方法定义：



ThreadLocal 实现线程隔离的方法：简单来说就是因为每个线程里面都有一个 ThreadLocalMap

ThreadLocalMap 存有一个 Entry key 为 threadLocal 对象，value 为共享变量的值。

然后我们来看看底层是不是这么实现的，先来看看 set 方法

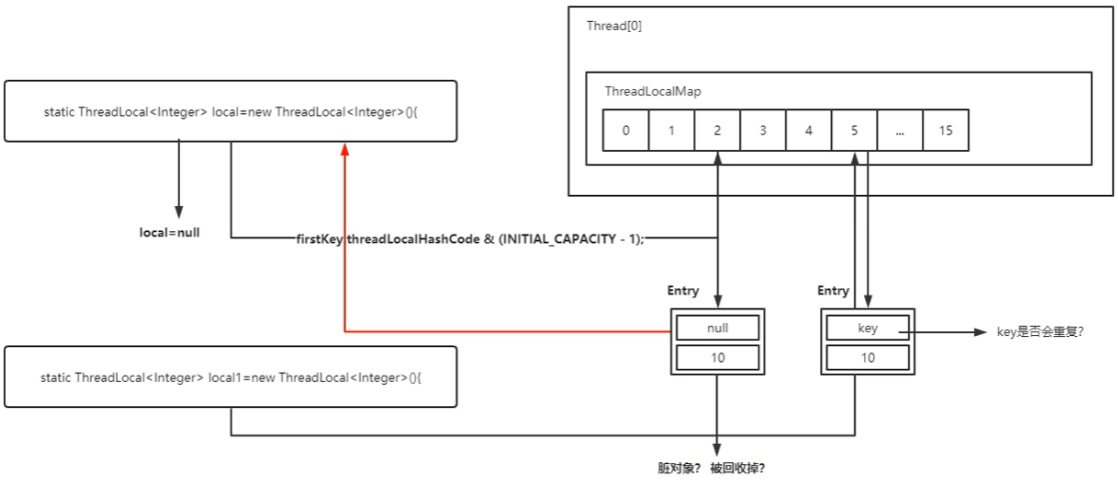
3.3 set 方法

```
public void set(T value) {
    Thread t = Thread.currentThread();//获取当前执行的线程
    ThreadLocalMap map = getMap(t);//获得当前线程的ThreadLocalMap 实例，
存有根据当前线程的ID 为Key 的ThreadLocalMap 里；
    if (map != null)//如果map 不为空，说明当前线程已经有了一个ThreadLocalM
ap 实例
        map.set(this, value);//直接将当前value 设置到ThreadLocalMap 中

    else
        createMap(t, value);//说明当前线程是第一次使用线程本地变量，构造ma
p，跟初始化的方法是一个
}


void createMap(Thread t, T firstValue) {
    //this 代表当前ThreadLocal 实例，firstValue 代表传进来的共享表量值
    t.threadLocals = new ThreadLocalMap(this, firstValue);
}
```

帅气的学长




```java
//初始化 ThreadLocalMap
ThreadLocalMap(ThreadLocal<?> firstKey, Object firstValue) {
    table = new Entry[INITIAL_CAPACITY];//
    int i = firstKey.threadLocalHashCode & (INITIAL_CAPACITY - 1);//thr
eadLocalHashCode
    table[i] = new Entry(firstKey, firstValue);
    size = 1;
    setThreshold(INITIAL_CAPACITY);
}

private final int threadLocalHashCode = nextHashCode();

private static final int HASH_INCREMENT = 0x61c88647;//魔数
private static int nextHashCode() {
    return nextHashCode.getAndAdd(HASH_INCREMENT);
}
```

上面的代码都是为了引出下面这个代码，证明 key 为弱引用

```java
static class Entry extends WeakReference<ThreadLocal<?>> {
    /** The value associated with this ThreadLocal. */
    Object value;

    Entry(ThreadLocal<?> k, Object v) {
        super(k);//key 为弱引用
        value = v;
    }
}
```

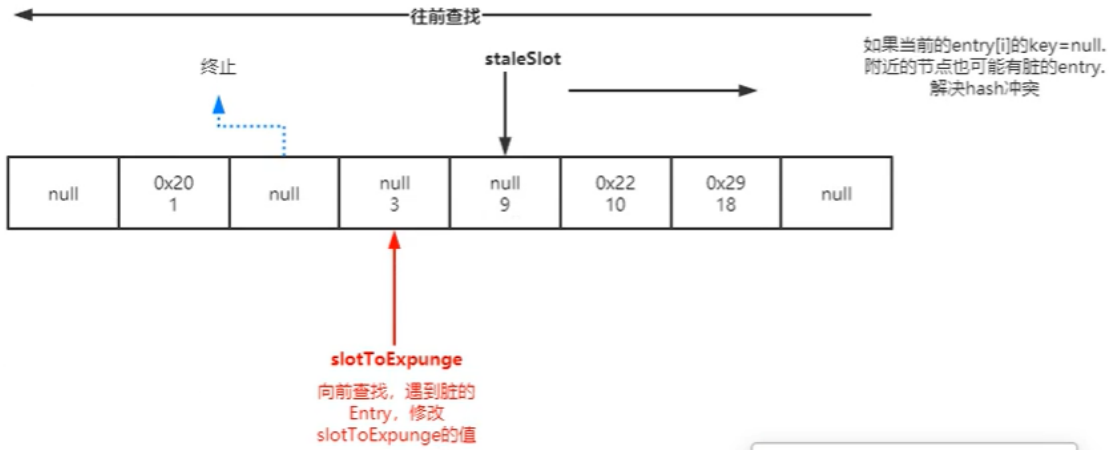
帅气的景天

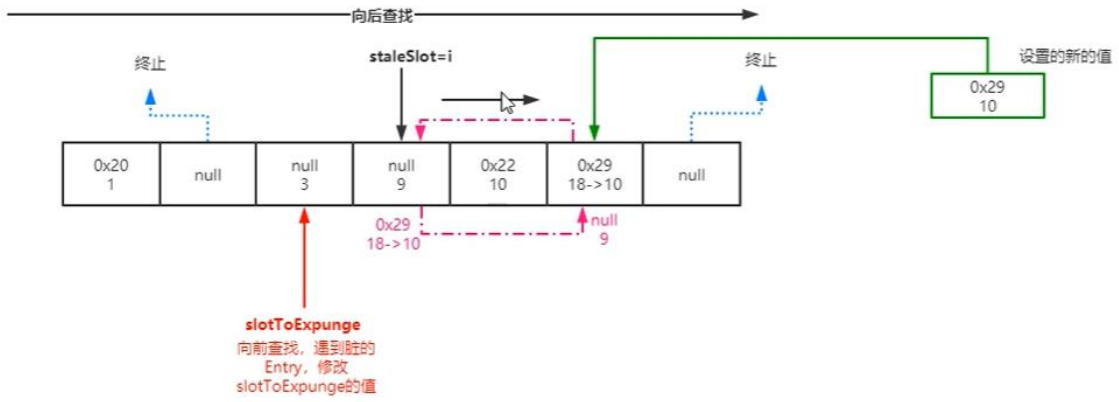
```java
private void set(ThreadLocal<?> key, Object value) {
    Entry[] tab = table;    //得到初始化好的16 个Entry 格子
    int len = tab.length;    //得到格子的长度
    int i = key.threadLocalHashCode & (len-1); //得到下标值，魔数，但是长度必须是2 的N 次方
    //进行线性探测，解决hash 冲突（默认e 是tab[i]，只要e!=null,都会执行 e = tab[i = nextIndex(i, len)]）,直到e 为空为止
    for (Entry e = tab[i];e != null;e = tab[i = nextIndex(i, len)]) {

        //拿到entry 的key 的值
        ThreadLocal<?> k = e.get();
        //如果等于我传进来的key,说明还没有被gc 回收，设置成功返回
        if (k == key) {
            e.value = value;
            return;
        }
        //如果key 已经被回收，调用replaceStaleEntry
        if (k == null) {
            replaceStaleEntry(key, value, i);
            return;
        }
    }
    tab[i] = new Entry(key, value);
    int sz = ++size;
    if (!cleanSomeSlots(i, sz) && sz >= threshold)
        rehash();
}
```

帅气的杀手

3.3.1 replaceStaleEntry






```
private void replaceStaleEntry(ThreadLocal<?> key, Object value,

                               int staleSlot) {
    //得到默认的Entry 数组以及长度
    Entry[] tab = table;
    int len = tab.length;
    Entry e;
    int slotToExpunge = staleSlot; //传进来当前的下标索引
    //向前扫描，查找最前的一个无效slot（即key为null 的entry）,并把下标给到
slotToExpunge
    for (int i = prevIndex(staleSlot, len);(e = tab[i]) != null;i = pre
vIndex(i, len))
        if (e.get() == null)
            slotToExpunge = i;
    //向后扫描不为空的entry
    for (int i = nextIndex(staleSlot, len);(e = tab[i]) != null;i = nex
tIndex(i, len)) {
        //得到entry 的key
        ThreadLocal<?> k = e.get();
        //找到了key,将其与无效的entry 交换
        if (k == key) {
            e.value = value;
            //相互交换，staleSlot 为最开始的时候的下标，e 为找到的不为空的en
try,i 为不为空的下标   减少hash 冲突，因为我现在的hash 是我的staleSlot

            tab[i] = tab[staleSlot];
            tab[staleSlot] = e;
            //进行分段清理
            if (slotToExpunge == staleSlot)
                slotToExpunge = i;
            cleanSomeSlots((slotToExpunge), len);
            //返回
            return;
        }
```

```
        if (k == null && slotToExpunge == staleSlot)
            slotToExpunge = i;
    }
    // If key not found, put new entry in stale slot
    //第一次，都找不到，在原有的位置，new Entry
    tab[staleSlot].value = null;
    tab[staleSlot] = new Entry(key, value);
    // If there are any other stale entries in run, expunge them
    if (slotToExpunge != staleSlot)
        cleanSomeSlots(expungeStaleEntry(slotToExpunge), len);
}
```

3.3.2expungeStaleEntry

```
/** * 这个函数是 ThreadLocal 中核心清理函数，它做的事情很简单： * 就是从 staleSlot 开始遍历，将无效（弱引用指向对象被回收）清理，即对应 entry 中的 value 置为 null，将指向这个 entry 的 table[i]置为 null，直到扫到空 entry。 * 另外，在过程中还会对非空的 entry 作 rehash。 * 可以说这个函数的作用就是从 staleSlot 开始清理连续段中的 slot（断开强引用，rehash slot 等） */
private int expungeStaleEntry(int staleSlot) {
    Entry[] tab = table;
    int len = tab.length;
    // expunge entry at staleSlot
    tab[staleSlot].value = null;
    tab[staleSlot] = null;
    size--;
    // Rehash until we encounter null
    Entry e;
    int i;
    for (i = nextIndex(staleSlot, len);
         (e = tab[i]) != null;
         i = nextIndex(i, len)) {
        ThreadLocal<?> k = e.get();
        if (k == null) {
            e.value = null;
            tab[i] = null;
            size--;
        } else {
            //对于还没有被回收的情况，需要做一次rehash。
            int h = k.threadLocalHashCode & (len - 1);
            if (h != i) {
                tab[i] = null;
                // Unlike Knuth 6.4 Algorithm R, we must scan until

                // null because multiple entries could have been stale.

                while (tab[h] != null)
                    h = nextIndex(h, len);
```

```
                tab[h] = e;
            }
        }
    }
    return i;
}
```

前面分析了 set 方法第一次初始化 ThreadLocalMap 的过程，也对 ThreadLocalMap 的结构有了一个全面的了解。那么接下来看一下 map 不为空时的执行逻辑

- 根据 key 的散列哈希计算 Entry 的数组下标
- 通过线性探索探测从 i 开始往后一直遍历到数组的最后一个 Entry
- 如果 map 中的 key 和传入的 key 相等，表示该数据已经存在，直接覆盖
- 如果 map 中的 key 为空，则用新的 key、value 覆盖，并清理 key=null 的数据
- rehash 扩容

3.4 get 方法

```
public T get() {
    Thread t = Thread.currentThread();
    ThreadLocalMap map = getMap(t);
    if (map != null) {
        ThreadLocalMap.Entry e = map.getEntry(this);
        if (e != null) {
            @SuppressWarnings("unchecked")
            T result = (T)e.value;
            return result;
        }
    }
    //初始化
    return setInitialValue();
}
```

3.4.1 getEntry

```
private Entry getEntry(ThreadLocal<?> key) {
    //根据ThreadLocal 的Id 来获取索引，也就是哈希值
    int i = key.threadLocalHashCode & (table.length - 1);
    Entry e = table[i];
    //这个ThreadLocal 有对应的entry 并且未失效（因为是弱引用）
    if (e != null && e.get() == key)
        return e;
    else
```

```
        //线性探测，所以往后找还是有可能能够找到目标Entry 的。e 为空的时候没用，还是返回的null,不为空的时候只有一种情况，entryKey 失效了
        return getEntryAfterMiss(key, i, e);
}
```

3.4.2 getEntryAfterMiss

```
//没有获取到e 的时候调用该方法
private Entry getEntryAfterMiss(ThreadLocal<?> key, int i, Entry e) {
    Entry[] tab = table;
    int len = tab.length;
    //不断向后探测直到遇到空entry。
    while (e != null) {
        //得到entry 的key,弱引用的调用方式
        ThreadLocal<?> k = e.get();
        //能拿到ThreadLocal 的值
        if (k == key)
            return e;
        if (k == null)
            //对应的ThreadLocal 已经被回收，调用expungeStaleEntry 来清理无效的entry
            expungeStaleEntry(i);
        else
            //不为空，拿到下一个下标
            i = nextIndex(i, len);
        //拿到下一个Entry
        e = tab[i];
    }
    return null;
}


//环形，不会超过len
private static int nextIndex(int i, int len) {
    return ((i + 1 < len) ? i + 1 : 0);
}
```

3.4.3 setInitialValue

```
private T setInitialValue() {
    //value 默认为空
    T value = initialValue();
    //得到当前线程
    Thread t = Thread.currentThread();
    //当前线程是否存在ThreadLocalMap
    ThreadLocalMap map = getMap(t);
```

帅气的余天

```
    if (map != null)
        //如果已经存在ThreadLocalMap，threadLocal 为key,value 为null 赋值
给entry
        map.set(this, value);
    else
        //不存在，创建map
        createMap(t, value);
    return value;
}
```

至此，整个 get 方法完成

3.5 为什么 Key 要弱引用？

假如每个 key 都强引用指向 ThreadLocal 的对象，也就是下图虚线那里是个强引用，那么这个 ThreadLocal 对象就会因为和 Entry 对象存在强引用关联而无法被 GC 回收，造成内存泄漏，除非线程结束后，线程被回收了，map 也跟着回收。

如果 key 是强引用，那么当我们执行 threadLocal=null 时，这个对象还被 key 关联，无法进行回收，只有当线程结束后，才会取消关联

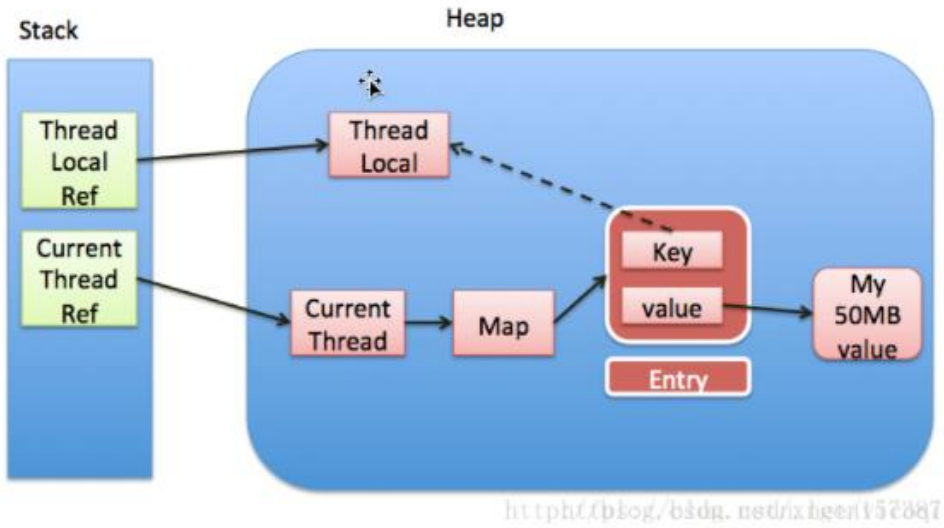
但是用弱引用，我们就能在 GC 的时候，回收！

但是如果用的是线程池，那么的话线程就不会结束，只会放在线程池中等待下一个任务，但是这个线程的 map 还是没有被回收，它里面存在 value 的强引用，所以会导致内存溢出。

所以一般用 threadLocal.remove()来清除内存

帅气的景天

1、在ThreadLocal的生命周期中，都存在这些引用。看下图：**实线代表强引用，虚线代表弱引用。**



3.6 Value 为什么不用弱引用

是因为不清楚这个 `Value` 除了 `map` 的引用还是否还存在其他引用，如果不存在其他引用，当 `GC` 的时候就会直接将这个 Value 干掉了，而此时我们的 `ThreadLocal` 还处于使用期间，就会造成 Value 为 null 的错误，所以将其设置为强引用

key 在，value 不在了

3.7 怎么解决 hash 冲突

1.首先，那个魔数就能保证重复性会低，但是基数必须是 2 的 N 次方（举例）

2.用开放寻址法，如果真的查到的下标已经存在数据，就去找下一个，找到一个 null 的为止，并且是环形查找，因为肯定会有空的，会进行提前扩容

3.8 引用类型

除了基础的数据类型外都是引用类型，那么 java 根据其生命周期的长短又将引用类型分为强引用、软引用、弱引用、虚引用

3.8.1 强引用

也是我们平时用得最多的，new 一个对象就是强引用，例如 Object obj = new Object();

当 JVM 的内存空间不足时，宁愿抛出 OutOfMemoryError 使得程序异常终止也不愿意回收**具有强引用的存活着**的对象！记住是存活着，不可能是你 new 一个对象就永远不会被 GC 回收。如果将引用赋值为 null 时，你的对象就表明不是存活着，这样就会可以被 GC 回收了

- 当内存不足的时候，jvm 开始垃圾回收，对于强引用的对象，就算出现 OOM 也不会回收该对象的。
  因此，强引用是造成 java 内存泄露的主要原因之一。

```
public static void main(String[] args) {
      Object obj=new Object();//这样定义就是一个强引用
      Object obj2=obj;//也是一个强引用
      obj=null;
      System.gc();
      //不会被垃圾回收
      System.out.println(obj2);
   }
```

3.8.2 软引用

软引用的生命周期比强引用短一些。软引用是通过 SoftReference 类实现的。当 JVM 认为内存空间不足时，就会去试图回收软引用指向的对象

对于只有软引用的对象来说，
当系统内存充足时，不会被回收；
当系统内存不足时，会被回收；

```
Object obj=new Object();
SoftReference wrf=new SoftReference(obj);
obj=null;
System.out.println("未发生 GC 之前"+wrf.get());
System.gc();
System.out.println("内存充足，发生 GC 之后"+wrf.get());
```

3.8.3 弱引用

弱引用是通过 WeakReference 类实现的，它的生命周期比软引用还要短，也是通过 get()方法获取对象。在 GC 的时候，不管内存空间足不足都会回收这个对象，同样也可以配合 ReferenceQueue 使用，也同样适用于内存敏感的缓存。ThreadLocal 中的 key 就用到了弱引用。

冰冷的景天

```
Object obj=new Object();
WeakReference wrf=new WeakReference(obj);
obj=null;
System.out.println("未发生 GC 之前"+wrf.get());
System.gc();
System.out.println("内存充足，发生 GC 之后"+wrf.get());
```

3.8.4 虚引用

也称虚引用，是通过 PhantomReference 类实现的。任何时候可能被 GC 回收，就像没有引用一样。无法通过虚引用访问对象的任何属性或者函数。那就要问了要它有什么用？虚引用仅仅只是提供了一种确保对象被 finalize 以后来做某些事情的机制。比如说这个对象被回收之后发一个系统通知啊啥的。

虚引用是必须配合 ReferenceQueue 使用的，具体使用方法和上面提到软引用的一样。主要用来跟踪对象被垃圾回收的活动。

如果一个对象只是被弱引用引用者，那么只要发生 GC，不管内存空间是否足够，都会回收该对象。

帅气的景天

4 面试题

1）sleep , join() /yiled() 的区别

sleep 让线程睡眠指定时间, 会释放 cpu 时间片（不会释放锁）

join， wait/notify， 让线程的执行结果可见

yiled 让出时间片. -> 触发重新调度.

sleep(0) -> 触发一次切换

2）Java 中能够创建 volatile 数组吗？

可以创建, Volatile 对于引用可见，对于数组中的元素不具备可见性。

// volatile 缓存行的填充. ->性能问题

3）Java 中的++操作是线程安全的吗？

不是线程安全的， 原子性、有序性、可见性。

++操作无法满足原子性

4）线程什么时候会抛出 InterruptedException()

t.interrupt() 去中断一个处于阻塞状态下的线程时（join/sleep/wait）

5）Java 中 Runnable 和 Callable 有什么区别

6）有 T1/T2/T3 三个线程，如何确保他们的执行顺序

join

7）Java 内存模型是什么？

JMM 是一个抽象的内存模型。

它定义了共享内存中多线程程序读写操作的行为规范：在虚拟机中把共享变量存储到内存以及从内

存中取出共享变量的底层实现细节。通过这些规则来规范对内存的读写操作从而保证指令的正确性，它解决了 CPU 多级缓存、处理器优化、指令重排序导致的内存访问问题，保证了并发场景下的

可见性。

8）什么是线程安全

原子性、有序性、可见性（硬件层面

（CPU 高速缓存、指令重排序、JMM））

9）volatile 能解决哪些安全问题

可见性和有序性

10）synchronized 是非公平锁，lock 可公平可非公平，他们都是重入锁

沸点的景秀